Web Application Penetration Testing Report

Document Title: Web Application Penetration Test DVWA on Metasploitable 2 **Report Version:** 1.0 Final **Classification:** Confidential Lab Exercise **Prepared By:** Rushil Patel **Date of Testing:** 17/05/2026 to 26/05/2026 **Report Date:** 26/05/2026

Target Application: Damn Vulnerable Web Application (DVWA) **Target Host:** Metasploitable 2 **Target IP:** 192.168.56.129 **Attacker IP:** 192.168.56.128 (Kali Linux) **Environment:** Isolated VMware Host-Only Lab Network **Authorization:** Self-authorized tester owns all systems

---

> [WARNING] LEGAL NOTICE This penetration test was conducted exclusively within a self-contained isolated lab environment. All target systems are owned and operated by the tester. No real systems, production environments, public IP addresses, or third-party services were accessed or tested at any point. All activities comply with ethical hacking principles.

1. Executive Summary

A web application penetration test was conducted against the Damn Vulnerable Web Application (DVWA) hosted on Metasploitable 2, running within an isolated VMware lab environment. The assessment was performed to identify, exploit, and document security vulnerabilities across the application's authentication, input handling, session management, and cookie configuration.

Overall Risk Rating: CRITICAL

The assessment identified 9 vulnerabilities across 5 categories. Two findings were rated Critical severity, three were rated High, and four were rated Medium or Low. The combination of SQL Injection, persistent Cross-Site Scripting, weak authentication controls, and insecure session management creates a chained attack path that could result in complete application compromise, mass account takeover, and full database exfiltration.

Key Findings at a Glance

Severity	Count	Examples
Critical	2	SQL Injection, Stored XSS
High	3	Reflected XSS, Weak Auth, Insecure Cookies
Medium	3	Missing CSP, X-Frame-Options, CSRF
Low	1	Server Version Disclosure
Total	**9**	

Critical Attack Chain Identified

The most severe risk identified was a chained attack path:

1. Stored XSS payload injected via guestbook (Finding 3)

2. Missing HttpOnly flag allows JavaScript to read session cookie
3. Every site visitor's session token silently exfiltrated
4. Stolen token replayed to gain authenticated access (confirmed)
5. SQLi then used to dump full database no password required

This chain demonstrates that individual vulnerabilities compound into catastrophic outcomes when multiple weaknesses coexist.

Immediate Actions Required

1. Implement parameterized queries for all database interactions
2. Apply output encoding to all user-supplied content
3. Set HttpOnly, Secure, and SameSite flags on all cookies
4. Enforce account lockout and rate limiting on login forms
5. Regenerate session IDs after successful authentication

2. Scope of Assessment

In-Scope Targets

Component	Details
Target IP	192.168.56.129
Application	DVWA (Damn Vulnerable Web Application)
Base URL	http://192.168.56.129/dvwa/
Modules Tested	SQL Injection, XSS Reflected, XSS Stored, Brute Force
Network	VMware Host-Only (VMnet1) isolated
OS	Metasploitable 2 (Ubuntu Linux 8.04)
Web Server	Apache 2.2.8
Database	MySQL 5.0.51a
Language	PHP 5.2.4

Out-of-Scope

- Any systems outside the 192.168.56.0/24 lab network
- The VMware host operating system
- Any internet-facing systems
- Any third-party services or APIs
- Real user accounts or data

Testing Constraints

- Security level maintained at Low for vulnerability demonstration
- All payloads were non-destructive (no data deletion/modification)
- Testing performed during controlled lab sessions only

3. Rules of Engagement

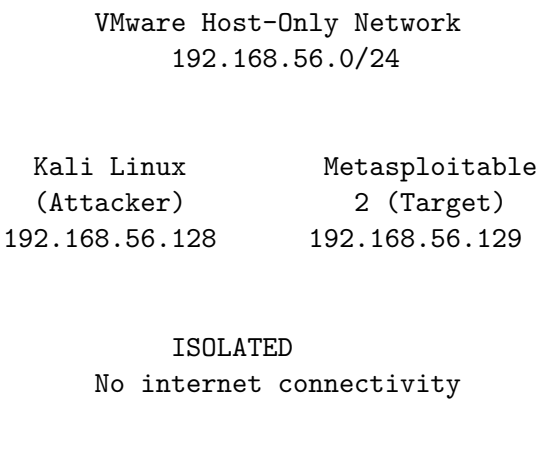
Rule	Detail
Authorization	Tester owns all systems fully authorized
Network boundary	VMware Host-Only only no internet traffic
Data handling	No real credentials or PII involved
Destructive testing	Not permitted read-only exploitation only
Denial of Service	Not tested out of scope
Escalation	N/A lab environment
Evidence retention	All screenshots and logs retained
Report distribution	Personal/portfolio use only

Ethical Commitments

- No attacks performed outside the defined lab network
- No real user data collected or retained
- No third-party systems contacted during testing
- All tools used within legal boundaries for education

4. Lab Architecture

Network Diagram



System Specifications

Attacker Machine Kali Linux - OS: Kali Linux (Rolling Release) - IP: 192.168.56.128 - RAM: 4 6GB allocated - CPU: 4 cores allocated - Role: Penetration testing platform

Target Machine Metasploitable 2 - OS: Ubuntu Linux 8.04 LTS - IP: 192.168.56.129 - RAM: 1GB allocated - CPU: 1 core allocated - Web Server: Apache 2.2.8 - Database: MySQL 5.0.51a - Application: DVWA (security=low) - Role: Intentionally vulnerable target

5. Tools Used

Tool	Version	Purpose	Phase
Nmap	7.x	Port scanning, service enumeration	3
Burp Suite Community	2023.x	Proxy, Repeater, Intruder	4,5,6,7,8
SQLMap	1.7.x	Automated SQL injection testing	4
Hydra	9.4	Brute force authentication testing	7
OWASP ZAP	2.x	Automated vulnerability scanning	9
Firefox DevTools	Built-in	Cookie and session inspection	8
Browser Console	Built-in	JavaScript cookie access test	8

Tool Justification

Nmap Industry standard for network reconnaissance. Used to map attack surface before application-level testing.

Burp Suite The professional standard for web app pentesting. Used across multiple phases for request interception, modification, and replaying demonstrating both manual skill and tool proficiency.

SQLMap Automated SQLi framework used to corroborate manual findings and demonstrate the scale of automated exploitation risk.

Hydra Command-line brute force tool demonstrating that authentication weakness is exploitable with widely available tools.

OWASP ZAP Free industry-standard automated scanner. Used to provide independent corroboration and surface additional header-based findings missed during manual testing.

6. Methodology

This assessment followed the OWASP Testing Guide (OTG) methodology and the Penetration Testing Execution Standard (PTES).

Testing Phases

RECONNAISSANCE

MANUAL VULNERABILITY IDENTIFICATION

EXPLOITATION & PROOF OF CONCEPT

AUTOMATED CORROBORATION (ZAP)

DOCUMENTATION & REPORTING

Phase Breakdown

Phase 1 Reconnaissance Active scanning using Nmap to enumerate open ports, identify running services, detect software versions, and fingerprint the operating system. Four scan types were used: basic, version, aggressive, and web-targeted.

Phase 2 Manual Vulnerability Testing Each DVWA module was tested methodically: - Baseline normal behavior established first - Incremental payload injection to confirm vulnerability - Impact demonstrated with escalating payloads - Evidence collected at each confirmation step

Phase 3 Tool-Assisted Exploitation Burp Suite used throughout for request capture, modification, and replay. SQLMap and Hydra used to demonstrate automated exploitation risk and scale.

Phase 4 Automated Scanning OWASP ZAP spider and active scan run against authenticated session. Results compared systematically against manual findings.

Phase 5 Documentation All findings documented with CVSS scores, OWASP mapping, evidence references, and actionable remediation guidance.

Security Testing Standard Reference

- OWASP Testing Guide v4.2
- OWASP Top 10 2021
- PTES (Penetration Testing Execution Standard)
- CWE (Common Weakness Enumeration)

7. Findings Summary

ID	Vulnerability	Severity	CVSS	OWASP 2021	Confirmed By
F1	SQL Injection	Critical	9.8	A03 Injection	Manual + SQLMap + ZAP
F2	Reflected XSS	High	7.4	A03 Injection	Manual + Burp + ZAP
F3	Stored XSS	Critical	8.2	A03 Injection	Manual + Burp
F4	Weak Authentication	High	7.5	A07 Auth Failures	Manual + Intruder + Hydra
F5	Insecure Session/Cookie Config	High	7.3	A07 Auth Failures	Manual + Burp + ZAP
F6	Missing CSP Header	Medium	5.4	A05 Misconfig	ZAP
F7	Missing X-Frame-Options	Medium	4.3	A05 Misconfig	ZAP
F8	Missing CSRF Token	Medium	4.3	A01 Access Control	ZAP
F9	Server Version Disclosure	Low	2.6	A05 Misconfig	ZAP + Nmap

Severity Distribution

Critical 2 High 3 Medium 3 Low 1

8.1 Finding F1 SQL Injection

Field	Detail
Severity	Critical
CVSS Score	9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
OWASP	A03:2021 Injection
CWE	CWE-89: SQL Injection
URL	http://192.168.56.129/dvwa/vulnerabilities/sqli/
Parameter	id (GET)
Tools	Manual, Burp Suite Repeater, SQLMap

Description

The `id` parameter accepts user input that is concatenated directly into a SQL query without sanitization or parameterization. An attacker can alter query logic to extract, modify, or delete any data in the connected database.

Proof of Concept

Step 1 Confirm injection with syntax break:

Input: ' Result: MySQL syntax error displayed confirms unsanitized input

Step 2 Extract all users with always-true condition:

Input: 1' OR '1'='1 Result: All 5 user records returned

Step 3 Extract database name:

Input: 1' UNION SELECT null,database()– - Result: dvwa

Step 4 Dump credentials:

Input: 1' UNION SELECT user,password FROM users– - Result: admin:5f4dcc3b5aa765d61d8327deb882cf99 (MD5 of 'password') gordonb:e99a18c428cb38d5f260853678922e03 [3 additional users]

Step 5 Automated confirmation (SQLMap):

sqlmap identified parameter 'id' as vulnerable Injection types: boolean-based blind, error-based, UNION query Database: dvwa | Tables: users, guestbook All credentials successfully dumped

Impact

- Full database content extracted without authentication
- All user credentials (including hashed passwords) exposed
- MD5 hashes trivially crackable with rainbow tables
- Potential for data modification or complete deletion
- In some server configurations: OS command execution possible

Evidence

- screenshots/sqli/02-single-quote-error.png
- screenshots/sqli/07-credentials-extracted.png
- screenshots/sqli/13-sqlmap-dump.png

Remediation

Priority: Immediate

1. **Parameterized Queries (Primary Fix):**

```php
// VULNERABLE:
$query = "SELECT * FROM users WHERE id = '$id'";

// SECURE:
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute([$id]);
```

2. Input validation whitelist numeric-only for ID fields
3. Least-privilege DB account read-only where possible
4. Disable detailed SQL error messages in production
5. WAF rules to detect and block common SQLi patterns

8.2 Finding F2 Reflected Cross-Site Scripting

Field	Detail
Severity	High
CVSS Score	7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N)
OWASP	A03:2021 Injection
CWE	CWE-79: Cross-Site Scripting
URL	http://192.168.56.129/dvwa/vulnerabilities/xss_r/
Parameter	name (GET)
Tools	Manual, Burp Suite Repeater

Description

The `name` parameter is reflected in the HTML response without output encoding. An attacker crafts a URL containing JavaScript and delivers it to victims. When clicked, the script executes in the victim's browser under the application's origin.

Proof of Concept

Basic execution:

```
Payload: <script>alert('XSS')</script>
Result:  Browser alert popup confirmed
URL:     http://192.168.56.129/dvwa/vulnerabilities/xss_r/
         ?name=<script>alert('XSS')</script>
```

Cookie theft simulation:

```
Payload: <script>alert(document.cookie)</script>
Result:  PHPSESSID=abc123; security=low displayed in alert
Impact:  In real attack, cookie sent to attacker-controlled server
```

Filter bypass via image tag:

```
Payload: <img src=x onerror=alert('XSS via img')>
Result:  Alert confirmed   demonstrates script-tag filter bypass
```

Raw HTML response (Burp Repeater):

```
<pre>Hello <script>alert(document.cookie)</script></pre>
```

Payload embedded unescaped directly in server response.

Attack Scenario

1. Attacker crafts URL with XSS payload
2. Delivers link via phishing email or social media
3. Victim clicks script runs in their browser
4. Session cookie exfiltrated to attacker server
5. Attacker replays cookie full account takeover

Impact

- Session hijacking via cookie theft
- Credential harvesting via injected fake login forms
- Malware delivery via browser redirects
- Reputation damage via visible page defacement

Evidence

- `screenshots/xss-reflected/02-alert-popup.png`
- `screenshots/xss-reflected/04-cookie-theft-simulation.png`
- `screenshots/xss-reflected/09-raw-html-payload.png`

Remediation

Priority: High

1. **Output encoding (Primary Fix):**

```php
// VULNERABLE:
echo "<pre>Hello " . $_GET['name'] . "</pre>";

// SECURE:
echo "<pre>Hello " . htmlspecialchars($_GET['name'],
     ENT_QUOTES, 'UTF-8') . "</pre>";
```

2. Content Security Policy header: `Content-Security-Policy: script-src 'self'`
3. HTTPOnly flag on session cookies (see Finding F5)
4. Input validation rejecting HTML special characters

8.3 Finding F3 Stored Cross-Site Scripting

Field	Detail
Severity	Critical
CVSS Score	8.2 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N)
OWASP	A03:2021 Injection
CWE	CWE-79: Cross-Site Scripting (Stored)
URL	http://192.168.56.129/dvwa/vulnerabilities/xss_s/
Parameters	txtName, mtxMessage (POST)
Tools	Manual, Burp Suite Repeater

Description

Unsanitized user input is stored in the MySQL database and rendered without encoding on every subsequent page load. Unlike reflected XSS, no victim interaction beyond a page visit is required all users who view the guestbook are automatically attacked.

Key Distinction from Reflected XSS

Factor	Reflected (F2)	Stored (F3)
Persistence	None one request	Database permanent
Victims	Must click crafted link	Every page visitor
Attacker presence	Required per victim	Not required
Severity	High	Critical

Proof of Concept

Payload submitted:

```
Name:    Attacker
Message: <script>alert(document.cookie)</script>
```

Confirmed persistence: - Alert fires immediately on submission - Alert fires again on every subsequent page visit - Alert fires for any user who visits the page - Payload persists in database until manually removed

Client-side control bypass: The HTML `maxlength` attribute on the Name field was bypassed using Burp Suite Repeater to send a POST request directly, confirming that client-side length restrictions provide zero server-side security.

Cookie exposure confirmed: `PHPSESSID=abc123xyz; security=low` displayed in alert popup.

Impact

- Mass session hijacking every visitor attacked automatically
- No social engineering required after initial payload submission
- Persistent attack survives server restarts until DB cleaned
- Client-side controls (maxlength) trivially bypassed via proxy

Evidence

- `screenshots/xss-stored/02-payload-submitted.png`
- `screenshots/xss-stored/03-alert-on-refresh.png`
- `screenshots/xss-stored/06-burp-bypass-charlimit.png`

Remediation

Priority: Immediate

1. Output encoding on all database-sourced content
2. Server-side input length validation never rely on maxlength
3. Content Security Policy header
4. Prepared statements prevent second-order injection
5. HTTPOnly cookie flag (see Finding F5)
6. Regular audit of stored user-generated content

8.4 Finding F4 Weak Authentication & Missing Brute Force Controls

Field	Detail
Severity	High
CVSS Score	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
OWASP	A07:2021 Identification & Authentication Failures
CWE	CWE-307: Improper Restriction of Excessive Auth Attempts
URL	http://192.168.56.129/dvwa/vulnerabilities/brute/
Parameters	username, password (GET)
Tools	Burp Suite Intruder, Hydra

Description

The login form implements no brute force protections. Combined with weak default credentials and GET-based credential submission, the application is trivially exploitable via automated password attacks.

Missing Controls Identified

Control	Status	Risk
Account lockout	Missing	Unlimited attempts
Rate limiting	Missing	Instant automated attacks
CAPTCHA	Missing	Bots not detected
POST method	Missing	Credentials in URL/logs
Strong password policy	Missing	Default ‘password’ accepted
MFA	Missing	Single factor only

Proof of Concept

Burp Suite Intruder:

Attack type: Sniper Wordlist: 10 passwords Target param: password Result: 'password' identified at attempt #9 Indicator: Response length 4924 vs 4891 for failures Time: Under 3 seconds

Hydra CLI:

```
hydra -l admin -P wordlist.txt 192.168.56.129 http-get-form
"...username=^USER^&password=^PASS^...:F=incorrect:H=Cookie:..."

Result: [80][http-get-form] login: admin   password: password
```

Credentials in URL:

http://192.168.56.129/dvwa/vulnerabilities/brute/?username=admin&password=password&Login=Login

Password visible in browser history, server access logs, and any network capture a separate vulnerability.

Impact

- Unlimited automated password attempts with zero detection
- Weak default credentials trivially discovered
- Credentials logged in plaintext in server access logs
- No mechanism to alert security team of attack in progress

Evidence

- `screenshots/weak-auth/07-intruder-attack-results.png`
- `screenshots/weak-auth/09-hydra-success.png`
- `screenshots/weak-auth/10-no-lockout.png`
- `screenshots/weak-auth/11-credentials-in-url.png`

Remediation

Priority: High

1. Account lockout after 5 failed attempts (15-30 min)
2. Rate limiting enforce delay between attempts per IP
3. CAPTCHA after 3 failures
4. Move to POST method over HTTPS immediately
5. Enforce strong password policy (min 12 chars, complexity)
6. Implement MFA renders brute force ineffective
7. Alert on repeated authentication failures

8.5 Finding F5 Insecure Session and Cookie Configuration

Field	Detail
Severity	High
CVSS Score	7.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
OWASP	A07:2021 Identification & Authentication Failures
CWE	CWE-1004, CWE-614, CWE-384
Scope	All application pages

Field	Detail
Tools	Browser DevTools, Burp Suite, Browser Console

Description

Session cookies are issued without critical security attributes, and the application fails to regenerate the session ID after login. Combined with XSS findings, this creates a complete session hijacking attack chain.

Vulnerabilities Identified

[A] Missing HttpOnly Flag

Set-Cookie: PHPSESSID=abc123; path=/

JavaScript can read the session cookie:

```
document.cookie      "PHPSESSID=abc123xyz; security=low"
```

Direct enabler for XSS-based session theft (Findings F2, F3).

[B] Missing Secure Flag Cookie transmitted over plain HTTP visible to any network observer performing traffic capture on the same segment.

[C] Missing SameSite Attribute Session cookie sent with all cross-site requests enables Cross-Site Request Forgery (CSRF) attacks.

[D] No Session Regeneration After Login

Pre-login PHPSESSID: abc123xyz Post-login PHPSESSID: abc123xyz Identical

Session fixation attack is possible attacker with pre-auth session ID gains post-auth access automatically.

[E] Client-Controlled Security Level

Cookie: security=low

Modified via Burp to `security=impossible` accepted by server. Application security must never rely on client-supplied values.

[F] Session Hijacking Demonstrated

1.Copied PHPSESSID from authenticated session 2.Pasted into separate private browser window via console 3.Navigated to DVWA authenticated as admin 4.No credentials entered

Secure vs Insecure Cookie Comparison

Attribute	Current	Required
HttpOnly	Missing	HttpOnly
Secure	Missing	Secure
SameSite	Missing	SameSite=Strict
Session Regen	Missing	After every login

Attribute	Current	Required
Expiry	Never	Max-Age=1800

Evidence

- `screenshots/cookies/03-javascript-cookie-access.png`
- `screenshots/cookies/06-burp-response-headers.png`
- `screenshots/cookies/09-session-fixation.png`
- `screenshots/cookies/10-session-hijack-poc.png`

Remediation

Priority: High

```
// Complete secure session configuration:
session_set_cookie_params([
    'lifetime' => 1800,
    'path'     => '/',
    'secure'   => true,
    'httponly' => true,
    'samesite' => 'Strict'
]);
session_start();

// Regenerate ID after login   prevents fixation:
session_regenerate_id(true);

// Never trust client-supplied security values:
// Remove security cookie   enforce server-side only
```

9. Remediation Summary

ID	Vulnerability	Priority	Fix Complexity	Primary Fix
F1	SQL Injection	Immediate	Low	Parameterized queries
F3	Stored XSS	Immediate	Low	htmlspecialchars()
F2	Reflected XSS	High	Low	htmlspecialchars()
F4	Weak Authentication	High	Medium	Lockout + MFA
F5	Insecure Cookies	High	Low	Cookie flags + session_regenerate_id()
F6	Missing CSP	Medium	Low	Add response header
F7	Missing X-Frame-Options	Medium	Low	Add response header
F8	Missing CSRF Token	Medium	Medium	CSRF token framework

ID	Vulnerability	Priority	Fix Complexity	Primary Fix
F9	Server Version Disclosure	Low	Low	ServerTokens Prod

Quick Win Headers (30 minutes to implement)

Add to Apache configuration or PHP header() calls:

Content-Security-Policy: default-src 'self' X-Frame-Options: DENY X-Content-Type-Options: nosniff Referrer-Policy: strict-origin-when-cross-origin ServerTokens Prod

Estimated Remediation Effort

- Critical findings (F1, F3): 1 2 developer days
- High findings (F2, F4, F5): 2 3 developer days
- Medium/Low findings: 1 developer day
- **Total estimated effort: 4 6 developer days**

10. Conclusion

This assessment identified 9 vulnerabilities across the DVWA application, including 2 Critical and 3 High severity findings. The application in its current state presents an unacceptable security risk and should not be deployed in any production environment without complete remediation.

Most Critical Risk Chained Attack Path

The combination of Stored XSS (F3), missing HttpOnly cookie flag (F5), and absence of session regeneration (F5) creates a fully-automated attack chain requiring no victim interaction beyond a page visit resulting in mass session hijacking and complete account takeover for all application users.

Lessons Learned

1. Vulnerabilities compound individual findings become catastrophic when combined into attack chains
2. Automated tools complement but cannot replace manual testing
   ZAP confirmed findings but missed exploitation depth that manual testing revealed
3. Defense in depth matters a single missing control (HttpOnly) transformed a contained XSS into mass credential theft
4. Client-side controls are not security controls every HTML restriction was bypassed via Burp Suite in minutes

Positive Observations

- Application correctly separated admin and user roles in DB
- PHP session IDs showed adequate randomness
- Apache configuration did not expose directory listings

Final Recommendation

Prioritize parameterized queries and output encoding as immediate fixes these two changes eliminate the two Critical findings (SQL Injection and Stored XSS) and significantly reduce overall risk. All remaining findings should be addressed within one sprint cycle.

11. Appendix

Appendix A Nmap Scan Outputs

See: `scans/nmap-basic.txt` See: `scans/nmap-version.txt` See: `scans/nmap-aggressive.txt`

Appendix B ZAP Automated Scan Report

See: `scans/zap-report.html`

Appendix C SQLMap Output

See: `scans/sqlmap-output/`

Appendix D Hydra Output

See: `scans/hydra-output.txt`

Appendix E CVSS Score Breakdown

ID	Vector String
F1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
F2	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N
F3	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N
F4	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
F5	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Appendix F References

- OWASP Top 10 2021: https://owasp.org/Top10/
- OWASP Testing Guide v4.2: https://owasp.org/www-project-web-security-testing-guide/
- PTES Standard: http://www.pentest-standard.org/
- CWE Database: https://cwe.mitre.org/
- CVSS Calculator: https://www.first.org/cvss/calculator/3.1

Appendix G Screenshot Index

See: `notes/evidence-index.md`